HATPro: Technical Overview and Architectural Summary

1. Purpose of This Document

This document provides a **concise, under-10-page summary** of the HATPro (Hospitality & Travel Profile) project for technical readers, standards participants, and implementers. It is intentionally **integrative rather than exhaustive**.

It explains: - what HATPro is (and is not), - how the data schema is designed and generated, - how the component libraries fit together, and - how the resulting artifacts support a **Traveler-controlled JSON profile**, optionally assisted by AI.

Detailed procedures, examples, and tooling specifics are **deliberately deferred** to the individual documents described in the *Reader's Guide to HATPro Development Docs*.

---

2. What HATPro Is (and Is Not)

2.1 What HATPro Is

HATPro is a **data-schema project** whose goal is to define a **portable, traveler-centric profile model** for use across the hospitality and travel ecosystem.

At its core, HATPro provides: - a modular **JSON Schema model** for a Traveler Profile, - generated **JSON templates** that can be used to instantiate profile data, and - a disciplined modeling and generation pipeline that ensures consistency, evolvability, and machine usability.

The Traveler Profile is intended to be: - **filled in by a traveler**, optionally with assistance from AI agents, - **shared selectively and purpose-by-purpose**, and - **independent of any single service provider or platform**.

2.2 What HATPro Is Not

HATPro is explicitly **not**: - an application or UI, - a storage or database schema, - a medical record system, - a booking or transaction protocol, - a verifiable-credential wallet (though it is designed to coexist with one).

HATPro focuses on **structured data definitions**, not runtime behavior or business logic.

---

3. Core Architectural Principle: Schema First

3.1 Data Schema as the Primary Artifact

The HATPro project treats the **data schema itself** as the primary and authoritative artifact.

Everything else—templates, examples, validation, documentation—derives from that schema.

This has several consequences: - Modeling decisions are made at the schema level, not buried in code. - The schema is stable enough to support long-lived profiles. - Generated artifacts are predictable and testable.

3.2 UML as the Authoring Language

All schemas originate from **UML models**, authored using **PlantUML**.

PlantUML is used because it: - is text-based and version-controllable, - supports modular includes, - allows precise control over structure without requiring heavyweight UML tooling.

Each significant schema component is represented by a **dedicated `.puml` file**, enforcing a one-class-per-file discipline.

---

4. From UML to JSON: The Generation Pipeline

4.1 Hints, Not Hand-Written Schemas

JSON Schema files are **not written by hand**.

Instead, HATPro uses a system of **explicit hints embedded in PlantUML notes** to drive code generation. These hints specify: - required vs optional properties, - enum binding, - structural constraints (e.g., XOR cases), - default values and ranges where applicable.

These hints act as **compiler directives**, not documentation prose.

4.2 Generated Artifacts

From the UML + hints, the pipeline produces:

1. **JSON Schema components**
   Modular, `$ref`-based schema files representing individual classes and enums.

2. **JSON Templates**
   Skeleton JSON documents that conform to the schemas and are suitable for:

   - direct manual completion, or

    - AI-assisted profile construction.
3. **Validation Outputs**
   Machine-verifiable confirmation that schemas, enums, and examples are consistent.

The generator and validation pipeline are described in detail in the developer documentation; this document focuses on the *shape and intent* of the outputs.

---

5. The Traveler Profile: Conceptual Structure

5.1 A Composed, Modular Profile

The Traveler Profile is **not monolithic**. It is composed from a set of modular components that can evolve independently.

At a high level, the profile consists of:

- **Core identity and contact data**
- **Shared preference and constraint libraries**
- **Travel-segment-specific components**

Each component is defined independently but composed into a single logical profile schema.

5.2 Core Components (Shared Across All Travel)

Core components represent information that is broadly applicable across travel contexts, including:

- **Identity data** (e.g., names, titles)
- **Addresses and locations**
- **Communication channels** (email, phone, messaging apps, priorities)
- **General preferences** (language, locale, currencies)

These components are intentionally generic and reusable.

They form a **shared library** that other profile segments depend on via references, rather than duplication.

---

6. Preferences, Constraints, and Support Needs

6.1 Declarative, Not Diagnostic

HATPro models preferences, constraints, and support needs as **declarative self-assertions**, not diagnoses.

Examples include: - mobility or accessibility needs, - sensitivities to environmental factors (altitude, temperature, noise), - food and substance exposure risks, - endurance or fatigue constraints.

The model captures **what matters operationally**, without attempting to explain *why*.

6.2 Graded vs Binary Expression

Where appropriate, the schema supports: - **binary declarations** (yes/no), and - **graded expressions** (ratings, ranges, tolerances).

This allows the model to start simple and evolve toward greater expressiveness without breaking structure.

Reusable primitives (e.g., rating scales, numeric ranges) are defined once in shared libraries and referenced wherever needed.

---

7. Travel-Segment-Specific Modeling

7.1 Segment Packages

In addition to shared libraries, HATPro supports **segment-specific packages** tailored to different parts of the travel experience, such as:

- lodging,
- air and rail transport,
- dining and food service,
- activities and attractions.

Each segment package: - reuses core libraries, - adds its own preferences and constraints, and - remains isolated from unrelated segments.

7.2 Why Segmentation Matters

Segmentation ensures that: - providers only request relevant data, - schemas remain comprehensible, - future extensions do not destabilize unrelated areas.

This aligns with the broader goal of **minimal, purpose-limited data sharing**.

---

8. JSON Templates and AI-Assisted Profile Creation

8.1 Templates as First-Class Outputs

Generated JSON templates are a key deliverable, not an afterthought.

They provide: - a concrete starting point for travelers, - a predictable structure for AI agents to work against, - a validation-safe way to incrementally build a profile.

8.2 Role of AI

AI is expected to assist with: - explaining fields in plain language, - suggesting values based on user input, - helping travelers complete complex sections.

Critically, AI operates **within the constraints of the schema**.

The schema defines what is allowed; AI helps populate it.

---

9. Governance, Evolution, and Stability

9.1 Separation of Structure and Semantics

HATPro intentionally separates: - **structural modeling** (schema shape), from - **interpretive or policy logic** (how data is used).

This makes the schema stable across jurisdictions, providers, and policy regimes.

9.2 Evolution Without Refactoring

The modular design allows: - adding new components, - enriching existing ones, - refining hints and generation rules,

without forcing wholesale rewrites of existing profiles.

Backward compatibility is a design goal, not an afterthought.

---

10. How to Use This Document

This overview should be read as: - a **conceptual map** of the HATPro system, and - a bridge between high-level intent and detailed developer documentation.

For specifics—file layouts, hint syntax, validation rules, migration plans—the **Reader's Guide** points to the authoritative documents.

Together, this overview and the Reader's Guide provide both: - the *why*, and - the *where to look next*.

11. Summary

HATPro defines a disciplined, schema-first approach to traveler profile data:

- authored in UML,
- generated into modular JSON Schema and templates,
- structured as reusable libraries plus segment-specific extensions,
- designed for traveler control and AI-assisted completion.

The result is a portable, extensible Traveler Profile model that can support a wide range of hospitality and travel use cases without locking data—or travelers—into any single system.